

Pipelines and feature selection for churn classification

1. What You'll Learn in This Section

In this lesson, you'll learn to:

- Organise preprocessing, feature selection, training, and prediction in a pipeline.
- Scale numeric features and reuse training statistics when predicting test data.
- Distinguish redundant features from irrelevant noise features.
- Compare churn classifiers with feature importance, F1, and confusion matrices.

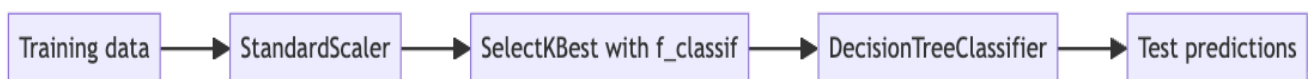
2. Detailed Explanation

Pipelines put model work in a declared order

A pipeline is a declared sequence of operations that runs together when it is fitted or used for prediction. It makes a workflow easier to read, reuse, and run again after a gap.

The pipeline does not give a model a new capability. Its value is consistency: it keeps the required steps together instead of leaving their order scattered across code.

A supervised-learning workflow can include cleaning unusable values, scaling numeric features, encoding values, selecting useful features, training a model, and predicting on test data. All preprocessing belongs before model training. The order within preprocessing depends on the data. For example, a developer might handle missing values before scaling, or remove an irrelevant column with missing values before scaling it.



Watch out for: A pipeline is the mechanism that orders operations. Preprocessing is the data-improvement work inside that sequence; they are not alternative ideas.

Scaling training data and predicting test data

StandardScaler scales numeric data so that the transformed data has mean 0 and standard deviation 1. It helps when numeric columns have different ranges and gives each numeric column a comparable scale.

A basic classification pipeline can name its steps `"scaler"` and `"model"`. The names are chosen by the developer. When the developer calls `fit(X_train, y_train)`, the scaler first learns the training-data mean and standard deviation. The model then trains on the transformed training data.

When the fitted pipeline receives `predict(X_test)`, it uses the statistics already learned from `X_train`. It does not calculate a new test-data mean. The fitted model then creates the test predictions.

Redundant features repeat information

A redundant feature repeats information that another feature already represents. Consider a game-prediction dataset with a numeric age and an age-related tag such as young, middle age, or higher age. Both describe closely related information, so keeping one can be sufficient.

Correlation helps identify this kind of repetition. A strong correlation means two features convey very similar information. A developer can then drop one of them. Age and an age-related tag can increase together, while age and height or age and gender can vary much more.

The game-prediction example can use age, height, gender, an age-related tag, and a school-standard column to predict games such as football, cricket, tennis, badminton, or basketball. Physical characteristics may support a prediction, but they are not strict rules. Practice and individual exceptions matter. A school-standard column may also fail to meaningfully determine the game when students have comparable opportunity to choose.

Redundant features mainly add complexity. They are usually less harmful than irrelevant features.

Irrelevant features create noise

An irrelevant feature does not meaningfully contribute to the target variable. It is also called a noise feature. An algorithm can search for a pattern in this kind of column and give it decision-making weight or feature importance.

Correlation between two feature columns does not establish irrelevance. Feature selection instead evaluates the relationship between an input feature and the class column. It asks whether that feature helps describe or predict the class values.

A synthetic churn dataset separates signal from noise

A synthetic churn dataset contains 150 customer records. Its three genuine inputs are `tenure_months`, `monthly_charges`, and `contract_month_to_month`. Tenure uses uniformly distributed values from 1 to 60; monthly charges use values from 20 to 120; and the contract feature is binary, generated with one binomial trial and probability 0.5.

The dataset also adds `noise_0` through `noise_5`. Each has 150 uniformly distributed values from 0 to 100. These six columns do not contribute to the churn outcome.

The churn target begins with a linear score `z`. Tenure has weight `-0.15`, monthly charges have weight `-0.03`, the month-to-month contract has weight `1.8`, and the constant is `2`. The logistic function converts that score into a churn probability:

```
probability_churn = 1 / (1 + e^(-z))
```

The logistic function produces a value from 0 to 1. A score of 0 gives 0.5; scores above 0 give probabilities above 0.5, and scores below 0 give probabilities below 0.5. Comparing that probability with a random value produces the binary label: 1 for churn and 0 for no churn.

The churn rate is the mean of the binary churn column. For example, $15 / 150 = 0.1$ means 15 churn cases among 150 records. This dataset has a churn rate of about 0.17, so about 17% of records are churn cases.

The weights are manually assigned for the synthetic design; they are not learned from an existing dataset. A negatively acting feature can still be useful because it affects the target. The six noise columns are irrelevant because none appears in the churn-generating score.

Split data while preserving class proportions

The input matrix `x` holds nine input columns: three genuine features and six noise features. The target `y` is the churn column. A train-test split places 30% of data in testing and 70% in training. For 150 records, that yields 105 training records and 45 test records.

Using `stratify=y` preserves each class proportion in both partitions. This matters for imbalanced classes. Without stratification, the test set can overrepresent one class while training poorly represents the other, even when the requested split percentage remains unchanged.

The declared pipeline sequence is independent of where the split appears in a notebook. Before `fit`, `x_train` and `y_train` must be available, because fitting runs every declared operation on training data. The same sequence can then be fitted again with new training data for the same problem setting.

Evaluate all features before refinement

A `DecisionTreeClassifier` makes feature importance visible through `feature_importances_`. With the Gini criterion, importance reflects each feature's contribution to reducing Gini impurity across the tree's decisions, including at the root or later levels.

In the all-features pipeline, the classifier follows scaling. The example achieves accuracy 0.844 and F1 score 0.462. Accuracy alone is misleading because no churn is the dominant class. F1 shows that the churn class is not detected reliably.

The tree's importance values sum to 1. Several noise columns receive nonzero importance even though three genuine columns determine the target. Together, the noise columns consume 41% of the importance share, creating unnecessary branching associated with the weak F1 score.

A horizontal bar chart can show the importance series. Reverse the feature names, values, and colours with `[:, -1]` before using `barh` when the most important feature should appear at the top.

Select the best features before training

SelectKBest performs feature selection before model training. With `f_classif`, it computes an ANOVA F-value between each input feature and the class column. A higher score means the feature is more useful for distinguishing the class values.

The refined pipeline uses three steps:

1. Scale input data with `StandardScaler`.
2. Select the top `k` features with `SelectKBest(f_classif, k=3)`.
3. Train the `DecisionTreeClassifier`.

The example uses `k=3` because it was designed with three genuine features. For an existing dataset, `k` is a hyperparameter: try and compare different values to find a setting that performs well. ANOVA F-values and chi-square tests offer a per-feature signal about the target relationship. They do not classify records; a machine-learning classifier combines multiple feature contributions to determine a class.

After refinement, F1 rises to `0.824`. Accuracy also rises, but the F1 change is more meaningful for the churn class. Logistic regression can also be compared before and after refinement, although it does not provide the decision-tree importance display.

Read confusion matrices with F1

A confusion matrix compares predicted labels with actual labels. The two matrices use `pred_all` and `pred_refined` against `y_test`, with labels `No churn` and `Churn`. They can appear as two subplots in a figure of size `9 by 4`, titled `All 9 features` and `Refined to 3`.

With all nine features, eight actual churn cases exist. The model predicts only three as churn and labels five as no churn. Those five are false negatives: churn cases predicted as no churn. After refinement, the model predicts seven of the eight churn cases as churn and predicts one as no churn.

A false positive is a no-churn case predicted as churn. A dominant no-churn class can keep accuracy high while the classifier misses many real churn cases. F1 exposes that weakness and shows the improvement after irrelevant features are removed.

Apply the workflow to a real dataset

For a real dataset with many features, build an appropriate pipeline, select useful features with a suitable score such as ANOVA F-values or chi-square, and compare model performance. There is no need to manually construct a noisy example.

One useful comparison changes `k` from `3` to `5` and examines how keeping the top five features affects model performance.

3. Key Takeaways

- Pipelines make preprocessing, feature selection, training, and prediction run in an explicit order.
- Standard scaling learns statistics from training data and reuses them for test predictions.

- Redundant features repeat information, while irrelevant features can add harmful noise.
- Stratification preserves class proportions, which matters when churn is less common.
- F1 and confusion matrices reveal churn-detection weaknesses that accuracy can hide.

Mental model: Think of feature selection as retaining the inputs that help distinguish churn from no churn before training the classifier.